Inside Liquorfy: A System Overview

Liquorfy compares alcohol prices across New Zealand liquor chains. You tell it where you are, and it tells you who has the bottle you want and what it costs nearby - priced so that a 6-pack and a 1L bottle are genuinely comparable. Sign in, and you can save a product and get an email the moment it drops below a price you've set or goes on special.

What follows is a look under the hood for anyone curious about how that's built. You don't need any prior context - this is the tour, not the manual, though it does go a level deeper than a pitch in the places that are genuinely interesting.

The Core Challenges

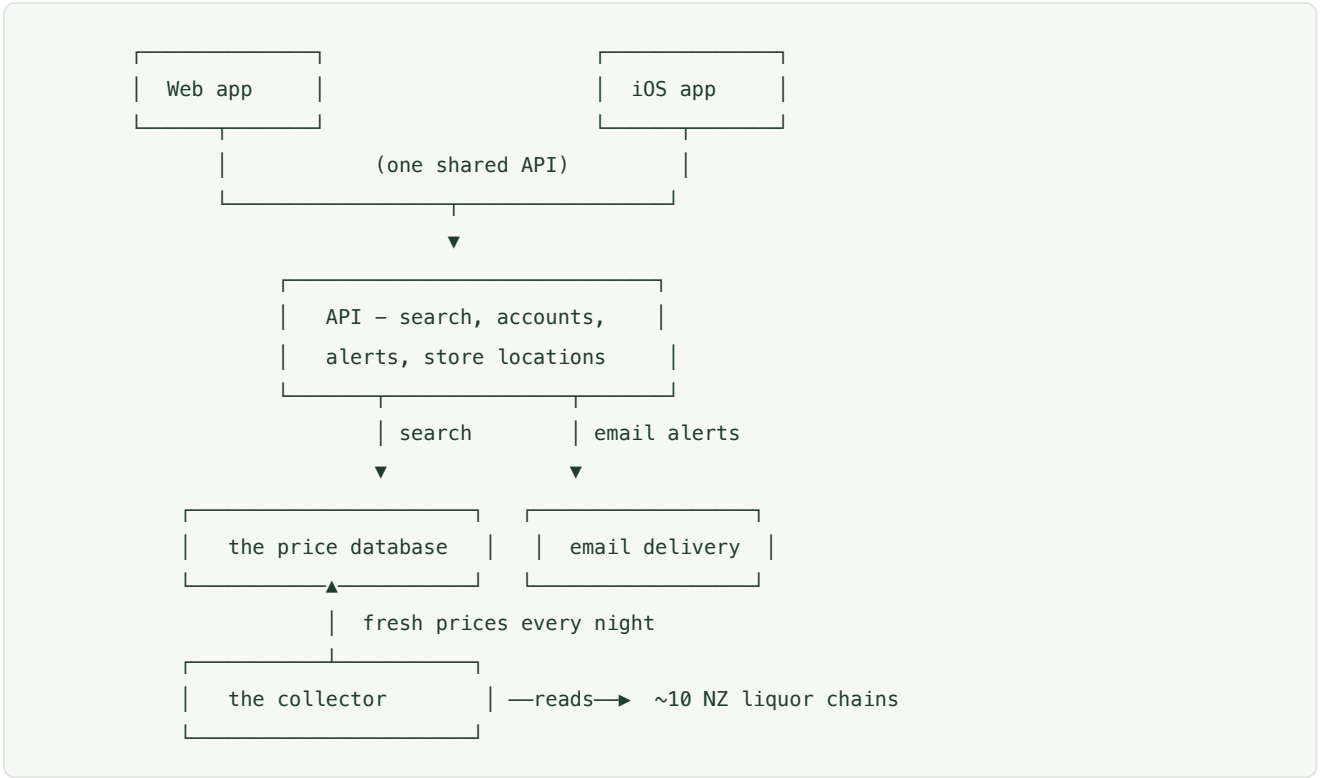
Putting prices on a page is the easy part. The real work is in everything behind it.

The first problem is that liquor chains don't share their prices. There's no tidy feed to plug into, so Liquorfy reads them directly from each chain's own website - around ten of them - every night.

The second is that no two chains describe a product the same way. One lists "Heineken Lager 12pk 330ml Bottles," another "HEINEKEN 12 X 330C BTL." Before you can compare prices, you have to work out that those are the same bottle.

The third is that prices and specials change constantly - and a wrong price is worse than no price at all. A special that ended yesterday but still shows today sends someone across town to the wrong shop.

Most of the engineering in Liquorfy goes into solving those three problems quietly, so that the app simply feels fast and correct.



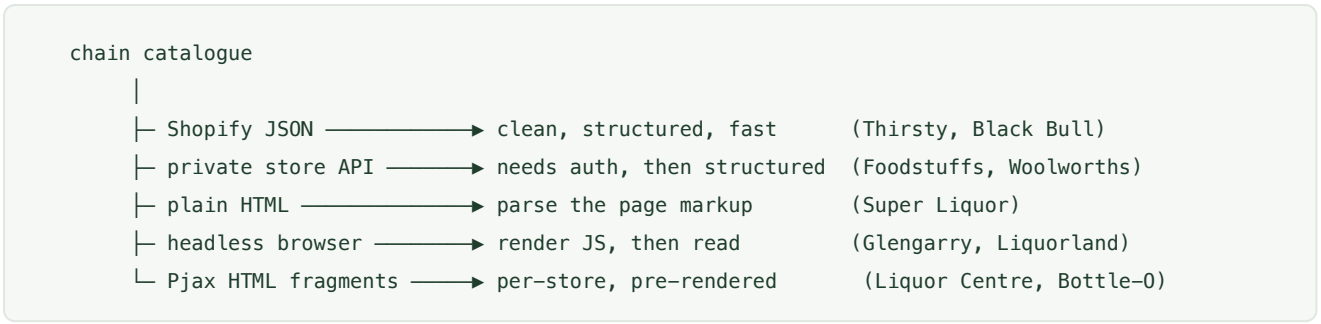
Collecting the Data

Every night, a background "collector" visits each chain's website and reads its catalogue - much like a very patient shopper scrolling through every product on the shelves. It works through one chain at a time so it never hammers anyone's site, and each chain takes anywhere from a few minutes to a couple of hours depending on how big the catalogue is and how hard the site is to read.

Four ways in

No two chains are scraped the same way, because no two chains publish their data the same way. In practice there are four strategies, picked per chain by whichever is fastest and most reliable:

- **Hidden JSON catalogues.** Some chains run on Shopify, which quietly exposes its full catalogue as clean JSON if you know the URL. This is the gold standard - structured, fast, no guessing. Thirsty Liquor and Black Bull are read this way, 250 products at a time.
- **Private store APIs.** The supermarkets (Pak'nSave, New World) and Woolworths/Countdown have internal APIs their own websites call. Liquorfy calls the same ones directly, which means no HTML parsing at all - but it first has to get past their front door (more on that below).
- **Reading the HTML.** Where there's no JSON to be found, the collector parses the visible web page itself, pulling prices out of the markup. Super Liquor works this way.
- **Driving a real browser.** A few sites render everything with JavaScript, so plain requests come back empty. For those, the collector launches a real headless Chrome (via Playwright) and reads the page after it renders. Glengarry and Liquorland need this.



That last one is a nice trick worth calling out. The franchise chains (Liquor Centre, Bottle-O) run on a platform called CityHive, where every store has its own subdomain. Those sites are JavaScript-heavy, but they'll hand back pre-rendered HTML fragments if you ask with the right header - so Liquorfy gets per-store pricing without paying the cost of a full browser for each of the ~90 stores.

Getting past the front door

The private store APIs don't just answer anonymous requests. Woolworths needs a valid session cookie, which the collector grabs with a quick ordinary request before it starts. The Foodstuffs supermarkets are fussier - they sit behind a bot-check and require an access token. The collector first tries a fast path (asking the site politely for a guest token over plain HTTP); if that's blocked, it falls back to launching a real browser, letting the bot-challenge resolve, and lifting the token out of the network traffic or the page's own storage. When a browser is needed at all, it runs with anti-detection measures and a real-looking user agent, locale, and timezone, so it behaves like an actual shopper in New Zealand rather than an obvious robot.

Being a polite guest

Scraping aggressively gets you blocked, so the collector is deliberately gentle. Browser-based runs wait about a second between pages and a couple of seconds between categories, retry failed pages up to three times with an ever-growing pause between attempts (two seconds, then four, then eight), and block images, fonts, and stylesheets from loading at all - the prices live in the text, so there's no reason to download the artwork. Long browser runs even close and reopen the browser every so often to keep sessions from going stale.

A couple of chains need extra cunning. Super Liquor's specials live on a different page from its regular catalogue, so the collector reads the specials first and then lets the regular pages fill in the proper "was/now" pricing. Liquorland publishes its specials through a completely separate promotions service, which Liquorfy reads and overlays on top of the baseline catalogue. And because category pages occasionally return nothing for a moment, the collector retries empty pages and gives up gracefully on a category only after several blanks in a row, rather than trusting the first zero.

Saving as it goes

Reading a few thousand products is a fragile business - a connection drops, or a site changes a page partway through a run. So the collector banks each page the moment it reads it, in its own

small database transaction. If a run fails three-quarters of the way through, everything it has already gathered is kept rather than discarded.

Under the hood, each page is written with an "insert or update" operation keyed on the chain and the chain's own product ID, so re-running a scrape never creates duplicates - it just refreshes what's there. Prices are written the same way, keyed on the product-and-store pair, and to stay within the database's limits they're flushed in batches of a couple of thousand at a time. Crucially, every write stamps the price with the current time ("last seen"), and whenever a price actually changes, the old value is also copied into a running history. Every run is recorded too: when it started and finished, how many items it touched, and whether it failed - which is what the health dashboard reads.

Same shop, different till

One subtlety the collector has to respect: some chains charge the same price at every location, while others - the franchises and supermarkets - vary their prices store by store. Liquorfy tracks which is which, so it never shows you one branch's special at another branch down the road. For the per-store chains, the collector walks each store individually and records a separate price for each.

Here's roughly how the ten chains break down:

Chain	How it's read	Pricing	Notes
Thirsty Liquor	Shopify JSON	Chain-wide	One price across 130+ stores
Black Bull	Shopify JSON	Per-store	Each franchise its own catalogue
Woolworths	Private API + cookie	Chain-wide	Searched by keyword (beer, wine, …)
Pak'nSave	Private API + token	Per-store	Walks every store; bot-check at the door
New World	Private API + token	Per-store	Same engine as Pak'nSave
Super Liquor	Plain HTML	Chain-wide	Specials read first, then overwritten
Glengarry	Headless browser	Chain-wide	JavaScript-rendered specialist retailer
Liquorland	Browser + promo feed	Chain-wide	Largest catalogue; ~2.5h runs
Liquor Centre	Pjax HTML, per store	Per-store	~90 CityHive store subdomains
Bottle-O	Pjax HTML, per store	Per-store	CityHive stores plus a franchise catalogue

When it all runs

A small scheduler wakes the collector at around 2am, once a day, and runs the chains one after another with a short gap between each. Each chain gets a time budget - two hours by default, but Liquorland is allowed five because its catalogue is so large - and if one overruns or crashes it's given a single retry rather than being allowed to block the rest. Before any of this starts, the scheduler checks the database is healthy and cleans up any "stuck" runs left behind by a previous crash (marking them failed so the dashboard stays honest). When the run finishes it posts a summary and triggers the price-alert check.

Normalizing and Matching Products

Raw product names are chaos: `STEINLAGER CLASSIC 12PK 330C BTL`. Before anything can be compared, Liquorfy untangles each one into real facts - the brand, the pack size, the volume, the alcohol percentage, and whether there's a special running. It even smooths over chain-specific quirks, like the CityHive sites that abbreviate "330ml" down to "330c."

Then comes the clever part. Liquorfy gives every product a fingerprint - a single value computed from the things that actually define the bottle: brand, variant, size, and pack count. The fingerprint is deliberately built to ignore the things chains disagree on, like how they categorise a drink or where they print the ABV, and to focus on what makes two bottles genuinely the same product. When two listings from two different chains reduce to the same fingerprint, they're recognised as the same thing. That single idea is what makes "this is $4 cheaper across town" possible at all. Without it, you'd have ten separate lists that never line up.

Ensuring Data Freshness

Prices go stale and specials expire, so Liquorfy is deliberately sceptical of its own data. Three independent safeguards work in concert, each catching a different failure.

A special vanishes without warning. Sometimes a promotion simply disappears from a chain's catalogue with no end date - it's just gone next time we look. So at the end of each run, the collector clears the promo fields on anything it *didn't* see this time around. It knows what it didn't see by comparing each price's "last seen" stamp against the moment the run began: anything older than that wasn't refreshed, so its special is no longer trustworthy. (This is done in batches of a thousand so it never overloads the database.)

A special quietly expires. Other promotions come with an end date. A routine cleanup job clears those out once the date has passed, independent of whether a scrape has run since.

A price simply goes stale. As a final backstop, the search itself distrusts any price the collector hasn't re-confirmed in the last seven days. Stale prices aren't deleted - they're just pushed down so they can never win a comparison on outdated information.

The combined effect is that what you see is current far more often than not - and when in doubt, Liquorfy would rather show you less than show you something wrong.

Location-Aware Search and How It Stays Fast

When you search, Liquorfy does more than match a name. It finds the shops around you that can legally sell alcohol - which neatly handles regional licensing quirks, such as parts of West Auckland - keeps only those within range, ranks the results by something genuinely useful (price per 100ml, or price per standard drink), and returns a page of them. Doing all of that in a fraction of a second, for many people at once, is its own engineering problem. A few ideas carry the weight.

The database is asked the right question. Rather than loading everything and filtering in code, the heavy lifting happens inside the database in a single query. The two expensive filters - "products whose name matches" and "stores near you" - are each isolated and explicitly told to use their dedicated index, so the database narrows down to a small candidate set before doing any real work.

Everything searchable is indexed for the way it's searched. Text matching is backed by a "trigram" index, which makes partial, case-insensitive name and brand matches fast instead of forcing a scan of every product. Location uses a geographic (PostGIS) index on each store's coordinates, so "within 5km of here" is a true spatial lookup. There are also targeted indexes on the things people sort and filter by - price, promo, recency - and a couple of *partial* indexes that only cover the rows that matter (for example, only products that have a cross-chain fingerprint, or only prices that actually have an expiry date), which keeps those indexes small and quick.

"Near me" is real geography, not a bounding box. Each store's location is stored as a proper geographic point. The radius filter uses a spatial "within this distance" test that the index can satisfy directly, and distance-sorting measures true on-the-ground distance rather than a crude rectangle. Coordinates use the standard GPS reference system, and the geographic point is computed automatically from each store's latitude and longitude.

Showing each product once, at its best price. When you ask for unique products, the query groups every listing of the same bottle together and, in a single pass, picks the cheapest, counts how many nearby stores carry it, and keeps only the best listing - all using database window functions rather than fetching everything and deduplicating in code. The "is there a live special?" decision is also made inside the query, by checking the promo price against its expiry on the spot.

Pages stay stable. Every sort has built-in tie-breakers, so two products that are equal on price don't randomly swap places between page one and page two - paging through results stays consistent. Page sizes are capped (at most 100 at a time) so a single request can never ask the database for an unbounded amount of work.

Common searches are remembered briefly. A fast in-memory cache (Redis) sits in front of the most repeated lookups - see the next section.

This matters because comparing a 6-pack to a single large bottle by sticker price tells you nothing. Normalising by how much you actually get is the entire point - and doing it quickly, at scale, is what makes the app feel instant.

Accounts and Price Alerts

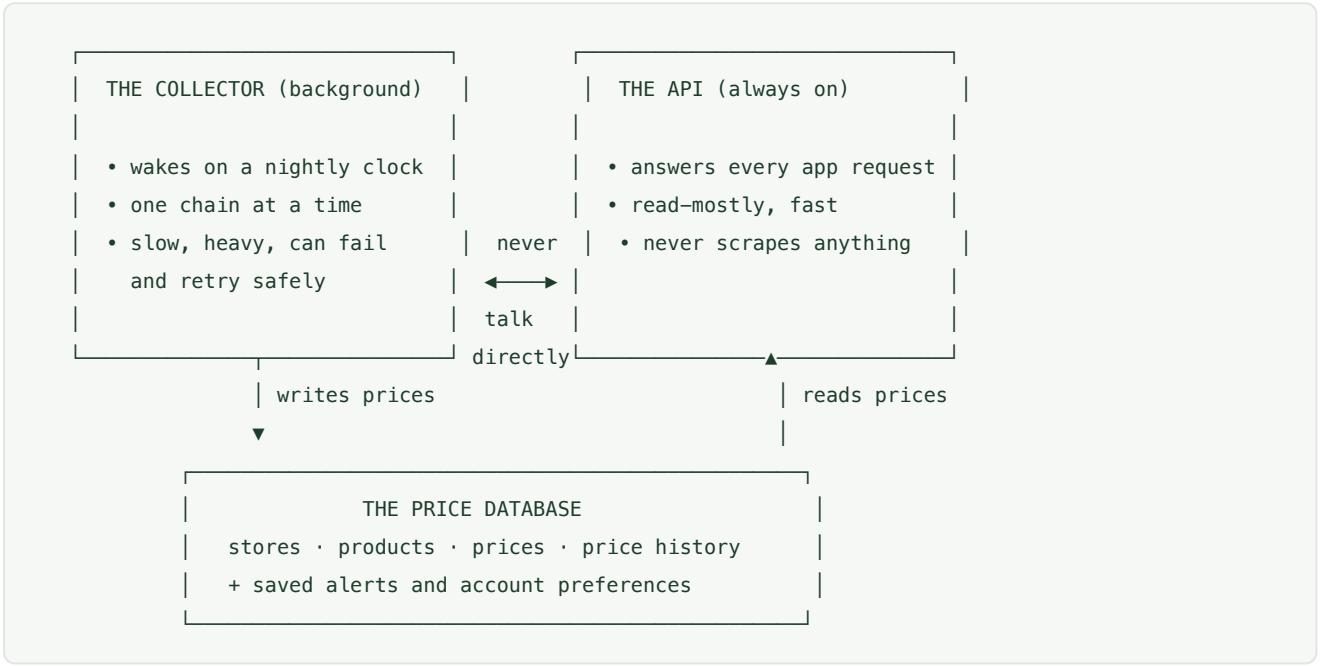
You shouldn't have to keep checking back, so you don't have to. Sign in, and you can save a product with a target - "let me know if this drops below $25," or simply "tell me when it's on special."

After each nightly collection, a watcher checks the latest best price for everything people are tracking. When a target is hit, it emails you the product, the price, the store, and a link - along with a

one-click unsubscribe. It also remembers the price it last told you about, so it won't nag you twice about the same drop.

Architecture: Collecting and Serving, Kept Apart

Looking one level deeper, the most important design decision is this: collecting prices and serving them are two entirely separate jobs that never talk to each other directly. They meet only through the database.



The reason for the split is that the two jobs have opposite personalities.

The collector is slow, heavy, and allowed to fail. A nightly run can take hours, drives real browsers, and routinely meets sites that have changed overnight. You want it isolated, so that when a scrape misbehaves it can crash, retry, and grind away without anyone noticing.

The API is the opposite: fast, always on, and never allowed to break. When you open the app, it simply reads already-collected prices from the database and responds in a fraction of a second. It never scrapes anything in the moment - all of that work was done hours earlier in the background.

Keeping the two apart means a struggling scrape can never slow down or take out the live app, and the live app can never interfere with collection. The database absorbs the difference between "gathered slowly overnight" and "served instantly on demand." Because they only meet at the database, each can be scaled, restarted, or rewritten without disturbing the other - and the API keeps a small pool of database connections open and reused, rather than paying to open a fresh one on every request.

The Data Model

It helps to understand how the data is shaped. There are really three things at the core:

- a **store** - a physical shop, with a location;

- a **product** - a specific bottle, with its fingerprint; and
- a **price** - simply the meeting of a product and a store: *this bottle, at this shop, costs this much, last seen on this date.*

That "last seen" date quietly does a lot of work - it's what the freshness checks read to decide whether a price can still be trusted. And every time a price changes, the old one is filed away as history, so the system can tell a genuine new low from a number that has merely been sitting unchanged. Around these three sit the lighter tables: saved alerts, account preferences, and the log of every collection run.

Anatomy of a Search Request

A request from either app follows a short, predictable path:

```
app  ──►  API: "lager near me, cheapest by volume"
              │  1. find shops near you that can sell alcohol
              │  2. match products to your search
              │  3. pull their latest prices, ignoring stale ones
              │  4. rank by price-per-volume, page the results
              ▼
app  ◄──  a ranked list — already collected, just assembled on the spot
```

Nothing in that path reaches out to a liquor chain. It's all reading and ranking data the collector banked earlier - which is precisely why it feels instant, even though gathering it took all night.

The Speed Layer: Redis

The main database is the source of truth, but it isn't always the fastest way to answer a question, and some questions get asked over and over. That's where Redis comes in - a small, in-memory data store that sits beside the database and remembers the answers to common requests for a short while.

It earns its place in two distinct ways.

The first is caching popular lookups. Most people in the same area are asking about the same patch of the map - "which stores are near me" resolves to the same answer for everyone in a suburb. So Liquorfy rounds each location into a rough bucket (to about a kilometre) and keeps the result for that bucket in Redis for ten minutes. The first person to ask pays the full cost of the geographic lookup; everyone nearby who asks soon after gets the cached answer back almost instantly. Because the entries expire quickly, the cache speeds things up without letting a stale answer linger once new prices land.

The second is handling sign-out securely. The apps stay signed in using a token that proves who you are on each request. These tokens are designed to be valid until they naturally expire, which raises an awkward question: what happens the moment you tap "log out"? Liquorfy keeps a short-lived blocklist in Redis. When you sign out, your token is added to it, and every request checks that list before trusting a token. The entry only needs to live until the token would have expired on its

own, after which it can be forgotten - which is exactly the kind of "remember this briefly, then drop it" job Redis is built for.

In short, Redis is the system's short-term memory. The database holds what is true; Redis holds what is worth keeping close at hand for the next few moments. If it ever went down, nothing would be lost - searches would simply recompute from the database, and the system would carry on a little slower.

Putting It Together

Two apps - web and iOS - share a single API. The web app is live today, while the iOS app is still in active development. The API answers searches, manages accounts, and sends alerts, leaning on Redis as a fast cache for common lookups and to handle sign-out. A background collector fills the database every night and keeps it honest. The database is where the two halves meet. The whole system runs as a small set of services that can be stood up together anywhere.